Movement Quest Project Report

CSC4360 Mobile App Development | Mon-Wed | Undergraduate

Project purpose

Movement Quest helps users plan short, single-exercise routines and record actual repetitions. It separates planned targets from completed work, allows corrections and preserves unfinished sessions on the device. The quest theme supports friendly progress feedback without rankings or medical claims.

Implementation and verification status

The source implementation covers the five proposed screen roles and two related SQLite entities. The local Dart helper evaluator and host SQLite schema audit have run successfully. Full Flutter analysis, Android compilation, device testing and team evidence remain release gates. This report describes the delivered implementation and the verified limits of the current evidence.

Existing responsibility assignments

Member	Proposal responsibilities
Mohammed Afeefuddin	Lead/UI owner: wireframes, navigation, forms, settings and demo coordination. Accessibility/integration review.
Muhammad Samad Khan	Data owner: SQLite, CRUD, activity persistence/integrity. Quality owner: helper evaluation, tests, documentation and release coordination.

These roles remain unchanged. Generated source does not establish either member's actual completed contribution. Task, commit, review and demonstration evidence must substantiate personal credit. AI development assistance is recorded separately in AI_Usage_Log.md.

Scope boundaries

One plan contains one exercise. Core behavior is local only: SQLite, SharedPreferences and on-device rules. There is no Firebase, cloud login, chatbot, live synchronization, wearable input or prediction of training intensity.

Screens and interaction

Role	Implemented behavior
S1 Catalog	Saved plan cards; create/open; expand/collapse; equipment filter; completed-session count; local suggestion with Apply/Not now.
S2 Editor	Labeled name, exercise/equipment controls, numeric bounds and personal intensity slider. Dirty-form cancel confirmation and pending-save guard.
S3 Detail/activity	Start/resume, log/correct sets, undo, finish, draft reset, keep/discard/stay on Back, plan edit/delete and history shortcut.
S4 History	Read/add/edit/delete completed sessions, parent filter, target snapshots and recomputed counts. Nonfuture date and set validation.
S5 Settings	Theme, equipment preference and reduced motion persisted through SharedPreferencesAsync. Retry keeps the last confirmed value.

Feedback and accessibility

Plan expansion uses 180 ms; saved-set feedback uses 220 ms; completed-workout feedback uses 320 ms. Optional transitions use zero duration when reduced motion is effective. Progress includes readable text. Forms and lists scroll, controls have labels, and destructive actions require confirmation. Portrait/landscape, enlarged text and screen-reader behavior still require actual device checks.

Error recovery

A failed save keeps the form or repetition input. Shared busy state blocks repeated writes. A debug-only failure switch rejects the next database operation to support a deliberate recovery demonstration. It is off in the normal release configuration. No historical bug or peer observation is invented.

Data and architecture

Two related entities

WorkoutPlan stores a name, exercise key, equipment category, target sets/repetitions, personal intensity label and timestamps. TrainingLog references a plan and stores a unique session token, draft/completed status, timestamps, target/exercise snapshots and actual repetitions as JSON. Snapshot fields preserve the original target after a plan changes.

Rule	Mechanism
Referential integrity	Foreign keys enabled on each connection; ON DELETE CASCADE; index on plan_id.
One draft per plan	Partial unique index plus transactional Start lookup/insert.
Duplicate actions	Disabled pending buttons, stable session token, expected-list-length check for set append, idempotent Finish.
Reversible progress	Edit replaces one position; Undo removes the last saved set; Reset deletes draft status only.
History integrity	All planned sets required for completion; reps 1-100; nonfuture manual dates; corrections update by ID.
Settings	One validated JSON preference object; theme/equipment/motion persist outside workout tables.

Separation and lifecycle

Screen widgets own local form state. QuestController owns shared data, busy/loading/error state and refresh. QuestRepository owns SQL and transactions. SettingsStore owns preference access. The ranking helper is pure Dart and takes a fixed clock for reproducible tests. Text controllers are disposed, async UI operations check mounted state, and the repository exposes awaited close.

Trade-off

ChangeNotifier and AnimatedBuilder keep the undergraduate implementation explicit. A larger app could use Provider or Riverpod and paged queries; these are future alternatives, not claims of implemented graduate requirements.

Local helper evaluation

Method: deterministic rule-based ranking. Score = 3 x exact equipment match - min(completed sessions in the preceding seven days, 2). Ties use lowest plan ID. No preference gives no equipment bonus. Baseline: lowest plan ID.

Evidence	Actual result
Execution	Pure Dart 3.13.5; fixed synthetic clock 2026-10-22T12:00:00Z.
Expected-output checks	10 of 10 passed. Normal, empty catalog, no history, two invalid cases, boundary, outside-boundary, tie, cap and counterexample.
Preference-label agreement	Helper 6/7; baseline 4/7 on the same seven valid synthetic cases. Three empty/invalid cases are excluded.
Scope of inference	Hand-authored demonstration fixtures. No held-out claim, user-study result, health assessment or general accuracy claim.

Counterexample and reasoning

A user can prefer a recently used bodyweight plan while the novelty penalty chooses another bodyweight plan. The evaluator deliberately records this mismatch. The ranking is mathematically correct for its rules but does not capture every user preference. The retained response is transparent reasoning and manual choice; a future familiarity preference would require new evaluation cases.

User control

Apply opens the plan without creating a session; Start is separate. Not now dismisses the suggestion. The user can always open a different plan. Invalid inputs suppress a suggestion with feedback. No history produces a disclosed preference-only ranking. The card explains that it considers all plans independently of the visible list filter.

Alternative

Nearest-neighbor matching on fictional labeled examples was considered. The explicit rules fit the small existing categorical data, need no training set and make each score inspectable. See AI_FEATURE.md and evidence/helper_results.json for complete details.

Testing and release gates

Check	Status and evidence
Helper evaluator	Executed: 10/10 output assertions; evidence/helper_results.json.
Actual SQL schema	Executed on host SQLite: 9/9 constraint checks; evidence/host_sql_schema_results.json. This does not test the Dart repository.
Dart source syntax	18 files parsed and formatted. No Flutter type-check claim.
Automated Flutter suites	14 repository, 10 helper, 2 widget and 1 integration case supplied; execution pending.
Android release/device	Build/install, two-device layouts, offline force-stop/restart and native settings checks pending.
Peer and collaboration	Real peer revision, missing board snapshots, task traces, reviews and personal reflections pending.

Challenges addressed in the implementation

Duplicate session creation is addressed with a unique draft index and transactions. Target drift is addressed with snapshots. Pending-save input loss is addressed by keeping the editor active until the write resolves. A deliberately rejected-write path makes error recovery reproducible. These are design/implementation choices, not claims about either member's historical debugging experience.

Before final submission

Generate the Android runner with the installed Flutter SDK, resolve dependencies and commit the lockfile, run analyze/tests, build and install the APK, and complete the two-device evidence matrix. Finish the Kanban PDF with real snapshots and individual traces. Add the approved proposal, official signed acknowledgment and accessible 20-25 minute YouTube walkthrough. Each student submits their own package.

Reflection and source materials

Each member must add their own concrete learning/reflection and contribution references from actual work. Requirements were taken from the supplied PROJECT DETAILS.docx and submitted proposal. Runtime implementation references: Flutter Material documentation, pub.dev/packages/sqflite and pub.dev/packages/shared_preferences. No missing individual evidence is represented as complete.